

CyberSOC Platform

Monitoring, Observability & Documentation Package

Phase 5: Go-Live Roadmap Execution

Document Type	Operations Documentation Package
Version	1.0.0 - Production Ready
Classification	Internal Operations
Date	2026-08-26
Scope	Full Observability Stack + Technical Documentation
Target Audience	SRE, DevOps, Support, Customer Success

Table of Contents

Part A: Observability Architecture

1. Observability Strategy Overview — Vision and guiding principles
2. Metrics Collection (Prometheus) — Infrastructure and application metrics
3. Visualization (Grafana Dashboards) — Operational dashboards by domain
4. Logging Architecture (Loki/ELK) — Centralized log aggregation
5. Distributed Tracing (Jaeger) — Request flow analysis
6. Alerting Framework — Rules, routing, and escalation
7. Runbook Library — Operational response procedures

Part B: Final Documentation Package

8. API Reference Documentation — Complete endpoint catalog
9. Administrator Guide — Configuration and management
10. User Guide — End-user operational procedures
11. Knowledge Base Articles — FAQ and troubleshooting
12. Release Notes Template — Version communication format

PART A: Observability Architecture

1. Observability Strategy Overview

The CyberSOC Platform observability strategy implements comprehensive visibility into system behavior through three complementary data types: metrics for quantitative monitoring of system health and performance, logs for discrete event recording enabling forensic analysis and debugging, and traces for distributed request tracking across microservice boundaries. This "three pillars" approach ensures operators possess appropriate tools for any investigation scenario whether identifying performance regressions through metric trend analysis, diagnosing specific failures through log correlation, or understanding complex request flows through trace visualization.

1.1 Observability Maturity Model

The observability maturity model defines progressive capability levels from basic monitoring through advanced predictive operations, providing a roadmap for continuous improvement investment prioritization. Current CyberSOC platform implementation targets Level 3 (Proactive) maturity across most domains with selected Level 4 (Predictive) capabilities in areas supporting AI-driven anomaly detection and capacity forecasting. The maturity assessment guides tool selection decisions, team skill development investments, and process automation priorities ensuring coherent advancement toward operational excellence objectives.

Level	Name	Metrics Capability	Logs Capability	Traces Capability
L1	Basic	UP/DOWN checks	Syslog collection	None
L2	Reactive	Resource utilization	Centralized search	Basic spans
L3	Proactive	Golden signals, SLOs	Structured, correlated	Full distributed
L4	Predictive	ML anomaly detection	Pattern recognition	Dependency mapping
L5	AI-Ops	Autonomous remediation	Intelligent parsing	Automatic root cause

Table 1.1: Observability Maturity Model Definition

2. Metrics Collection (Prometheus)

Prometheus serves as the primary metrics collection and time-series database for the CyberSOC platform, chosen for its cloud-native integration via native Kubernetes service discovery,

powerful PromQL query language enabling flexible metric analysis, and extensive ecosystem of exporters and integrations. The Prometheus deployment follows best practices including high-availability configuration with thanos long-term storage for retention beyond local node capacity, recording rule pre-computation for expensive queries, and alertmanager integration for notification routing to appropriate response teams based on alert severity and classification.

2.1 Metric Taxonomy and Naming Conventions

Consistent metric naming enables intuitive query construction and prevents naming collisions between components sharing similar measurement concepts. The naming convention follows Prometheus best practices using lowercase snake_case names with component prefix, measurement noun, and unit suffix structure. Labels provide query dimensionality without metric name proliferation, with cardinality limits enforced to prevent high-cardinality label combinations causing performance degradation. Standard labels include environment (production/staging/development), service name, instance identifier, and error type classifications enabling consistent filtering across all platform metrics.

Metric Name	Type	Labels	Description	Example Use
cybersoc_http_requests_total	Counter	method, status, path	HTTP request count	Error rate calculation
cybersoc_http_duration_seconds	Histogram	method, path, le	Request latency distribution	SLA compliance
cybersoc_events_processed_total	Counter	source, status	SIEM event throughput	Pipeline monitoring
cybersoc_active_incidents	Gauge	severity, status	Current incident count	Operational dashboard
cybersoc_db_connections_active	Gauge	database, pool	Database connection usage	Capacity planning
cybersoc_cache_hit_ratio	Summary	cache_name	Cache effectiveness	Performance tuning
cybersoc_threat_score	Histogram	threat_type, le	Threat scoring distribution	Detection calibration

Table 2.1: Standard Metric Definitions

2.2 ServiceMonitor Configurations

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: cybersoc-gateway
  namespace: cybersoc-monitoring
  labels:
    app.kubernetes.io/name: cybersoc-gateway
    release: prometheus-stack
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: cybersoc-gateway
  namespaceSelector:
    matchNames:
      - cybersoc-system
  endpoints:
    - port: metrics
      interval: 15s
```

```
scrapeTimeout: 10s
path: /metrics
honorLabels: true
relabelings:
- sourceLabels: [__meta_kubernetes_pod_node_name]
  targetLabel: node
metricRelabelings:
- sourceLabels: [__name__]
  regex: go_.*|process_.*
  action: drop
```

Listing 2.1: Gateway ServiceMonitor Configuration

3. Grafana Dashboards

Grafana provides visualization layer atop Prometheus metrics, presenting operational data through curated dashboards organized by operational domain and audience role. Dashboard design follows information hierarchy principles placing most critical indicators in prominent positions with progressive drill-down capability for investigation workflows. All dashboards include consistent time range controls, refresh interval settings, and annotation support for correlating metric behavior with deployment events, incident timelines, or external factors affecting system performance.

3.1 Dashboard Inventory

Dashboard ID	Name	Audience	Refresh Rate	Key Panels
DASH-001	Platform Overview	Exec/Management	5m	Availability, Error Rate, Latency P99
DASH-002	API Gateway Metrics	Platform Team	30s	RPS, Latency, Error Codes, SSL
DASH-003	Threat Engine Status	Security Ops	15s	Events/sec, Queue Depth, Scores
DASH-004	SIEM Pipeline Health	SIEM Team	30s	Ingest Rate, Correlation Latency
DASH-005	Database Performance	DBA Team	1m	Connections, Query Time, Locks
DASH-006	Kubernetes Cluster	SRE Team	1m	Nodes, Pods, Resources, Events
DASH-007	Incident Response	IR Team	1m	Open Incidents, MTTR, Severity
DASH-008	SLO Dashboard	SRE/Management	1m	Error Budget, Burn Rate, SLOs
DASH-009	Cost Allocation	Finance/Mgmt	1h	Cloud Costs, Resource Usage
DASH-010	Security Posture	Security Team	5m	Vulns, Auth Failures, Alerts

Table 3.1: Grafana Dashboard Inventory

3.2 SLO Dashboard Configuration

Service Level Objective (SLO) dashboards communicate reliability performance against defined targets to technical and business stakeholders alike. The dashboard displays multi-period error

budgets showing consumption rate trends predicting potential budget exhaustion before month-end if current burn rate continues. Error budget calculations incorporate both objective measurements (availability percentages from uptime monitoring) and subjective measurements (user-reported incidents classified as SLO violations). Dashboard annotations mark deployment events enabling rapid correlation between releases and SLO impact, supporting data-driven release decisions during critical periods approaching budget exhaustion.

4. Logging Architecture (Loki)

The logging architecture implements centralized log aggregation using Grafana Loki as the primary log storage and query engine, chosen for its cost-effective label-based indexing approach avoiding the full-text indexing expense of traditional ELK stacks while maintaining excellent query performance for structured log data. Fluent Bit agents deployed as DaemonSet on each Kubernetes node collect container logs, apply initial parsing transformations, and forward to Loki distributors for label extraction and chunk storage. Log retention policies balance investigative needs against storage costs with hot storage (SSD) retaining 30 days for active investigations, warm storage (HDD) extending to 90 days for compliance requirements, and cold object storage (S3) archiving to one year for forensic purposes.

4.1 Log Pipeline Architecture

Stage	Component	Technology	Function	Configuration
Collection	Agent	Fluent Bit DaemonSet	Log file tailing, metadata enrichment	parsers.conf
Parsing	Transformer	Fluent Bit filters	JSON extraction, regex patterns	filter-kubernetes.conf
Routing	Router	Fluent Bit outputs	Destination selection by label	outputs.conf
Ingestion	Distributor	Loki Distributor	Label validation, tenant routing	loki-distributed.yaml
Indexing	Ingester	Loki Ingester	Label index building	config.yaml
Storage	Store	Loki Store / S3	Chunk compaction, long-term	schema-config.yaml
Query	Query Frontend	Loki Query Frontend	Split, cache, merge queries	query-frontend-config.yaml
Visualization	UI	Grafana Explore	LogQL queries, correlation	Dashboard links

Table 4.1: Log Processing Pipeline Stages

4.2 LogQL Query Examples

```
# Example LogQL Queries for Common Investigations

# Find all 5xx errors from API gateway in last hour
{app="cybersoc-gateway"} |= `5\d{2}` | line_format "{{.timestamp}} {{.message}}"

# Count errors by HTTP status code
sum by (status_code) (count_over_time({app="cybersoc-gateway"} |= `"error"` [1h]))
```

```
# Trace request flow across services using trace_id
{trace_id="abc123"} |=`.*`

# Calculate error rate percentage
(
  sum(rate({app="cybersoc-gateway"} |~ `5\d{2}` [5m]))
  /
  sum(rate({app="cybersoc-gateway"} [5m]))
) * 100 > 1

# Find slow requests (> 2 seconds latency)
{app="cybersoc-gateway"} | json | duration > 2000

# Correlate authentication failures with source IPs
sum by (source_ip) (count_overtime({app="cybersoc-auth"} |~ `"login failed"` [1h])) > 10
```

Listing 4.1: Common LogQL Investigation Queries

5. Distributed Tracing (Jaeger)

Jaeger distributed tracing provides request-flow visibility across CyberSOC microservice boundaries, essential for diagnosing complex interactions where traditional per-service logging fails to reveal cross-component latency contributions or failure propagation paths. OpenTelemetry instrumentation libraries auto-capture span data for common frameworks (Express, FastAPI, gRPC) while custom instrumentation captures business-relevant operation boundaries such as threat detection pipeline stages or case management workflow transitions. Trace sampling strategies balance completeness against storage costs, applying tail-based sampling retaining all error traces while sampling a fraction of successful requests for baseline performance characterization.

5.1 Instrumentation Coverage Matrix

Service	Auto-Instrumentation	Custom Spans	Key Operations Traced	Sampling Rate
API Gateway	OTel HTTP middleware	Auth check, rate limit	Request routing, TLS termination	100% errors, 10% success
Auth Service	OTel HTTP + gRPC	Token validation, session	Logout, token refresh	100% auth ops, 5% other
Threat Engine	Custom Python OTel	Pipeline stages, ML inference	Event ingestion, scoring, alert gen	100% alerts, 1% normal
SIEM Core	Custom Python OTel	Correlation rules, alert gen	Event processing, search queries	100% slow (>1s), 0.5% normal
Case Management	OTel HTTP	CRUD operations, workflow	Create, update, assign, resolve	10% all operations
Frontend	OTel JS SDK	Page loads, API calls, user	Navigation, form submission, errors	5% user sessions

Table 5.1: Distributed Tracing Coverage by Service

6. Alerting Framework

The alerting framework transforms raw metric thresholds and log patterns into actionable notifications routed to appropriate responders through severity-classified channels. Alert design principles prioritize actionable alerts with clear runbook references over noisy threshold crossings lacking operational context. Each alert definition includes symptom description, likely cause hypotheses, immediate investigation steps, and escalation triggers preventing issue aging without appropriate attention. AlertManager handles deduplication, grouping related alerts into single notifications, inhibition rules suppressing downstream alerts when root cause is already acknowledged, and silence scheduling for planned maintenance windows.

6.1 Critical Alert Definitions

Alert Name	Expression	Severity	Channel	Response SLA
CyberSOCDown	up{job="cybersoc"} == 0	Critical	PagerDuty + Slack	< 5 min
HighErrorRate	error_rate > 5% for 5m	Critical	PagerDuty + Slack	< 10 min
LatencyP99High	histogram_quantile(0.99) > 5s	Warning	Slack + Email	< 30 min
DatabaseConnectionsExhausted	db_connections/limit > 0.9	Critical	PagerDuty + DBA	< 5 min
DiskSpaceCritical	disk_used_percent > 90%	Critical	PagerDuty + SRE	< 15 min
CertificateExpiringSoon	cert_expiry < 72h	Warning	Slack + Security	< 24h resolution
PodCrashLooping	kube_pod_container_status_restarts_total > 5	Warning	Slack + On-call	< 30 min
QueueBacklogGrowing	queue_length increasing > 10min	Warning	Slack + Team	< 1 hour
AuthFailureBurst	auth_failures_rate > 3x baseline	Warning	Slack + Security	< 15 min
SLOBurnRateFast	error_budget_remaining < 7 days	Critical	Slack + Management	< 1 day action

Table 6.1: Critical Alert Rule Definitions

7. Runbook Library

Runbooks document standardized response procedures for common operational scenarios, enabling consistent incident handling regardless of which team member responds and reducing mean time to resolution through proven step-by-step guidance. Each runbook includes trigger conditions defining when the procedure applies, diagnostic commands for situation assessment, decision trees branching based on diagnostic outcomes, remediation steps with expected results at each stage, verification commands confirming successful resolution, and escalation criteria triggering engagement of additional expertise or management notification. Runbooks undergo regular review incorporating lessons learned from recent incidents ensuring procedural currency with evolving system behavior.

7.1 Runbook Index

Runbook ID	Title	Category	Owner	Last Updated	Est. MTTR
RB-001	Application Pod CrashLoopBackOff	Kubernetes	Platform Team	2024-Q3	15 min
RB-002	High Memory Usage / OOMKills	Resources	SRE Team	2024-Q3	30 min
RB-003	Database Connection Exhaustion	Database	DBA Team	2024-Q2	20 min
RB-004	Elevated 5xx Error Rate	Application	On-call	2024-Q3	10 min
RB-005	SSL/TLS Certificate Expiry	Security	Platform Team	2024-Q3	5 min
RB-006	Disk Space Running Low	Infrastructure	SRE Team	2024-Q2	45 min
RB-007	Authentication Service Degradation	IAM	Auth Team	2024-Q3	15 min
RB-008	Message Queue Backlog Growing	Integration	Backend Team	2024-Q2	30 min
RB-009	Slow Database Query Detection	Database	DBA Team	2024-Q2	60 min
RB-010	Kubernetes Node Not Ready	Infrastructure	SRE Team	2024-Q3	45 min

Table 7.1: Operational Runbook Index

PART B: Final Documentation Package

This section outlines the complete documentation package required for General Availability (GA) release of the CyberSOC Platform. Documentation serves multiple audiences including end users performing daily security operations tasks, administrators configuring and maintaining platform deployments, developers integrating with platform APIs, and support engineers troubleshooting customer issues. All documentation adheres to consistent style guidelines, maintains accuracy through automated validation checks, and undergoes regular review cycles synchronized with software release cadence.

8. API Reference Documentation

The API reference documentation provides complete specification of all publicly accessible endpoints including request/response schemas, authentication requirements, rate limiting parameters, error code definitions, and example usage for each operation. Documentation generation leverages OpenAPI 3.0 specifications maintained alongside source code, ensuring synchronization between implementation and documentation through automated validation pipelines rejecting PRs where documented behavior diverges from actual implementation. Interactive API exploration through Swagger UI enables developer self-service testing without requiring local environment setup.

8.1 API Endpoint Catalog

Module	Endpoint Count	Base Path	Key Operations	Auth Required
Authentication	12	/api/v1/auth	Login, logout, MFA, tokens	Public subset
Threat Intel	25	/api/v1/threats	CRUD IOCs, feeds, search	Yes (API key)
SIEM Events	18	/api/v1/events	Ingest, search, correlate	Yes (JWT)
Incidents	22	/api/v1/incidents	Full lifecycle, comments	Yes (JWT)
Cases	20	/api/v1/cases	Workflow management	Yes (JWT)
Users/Admin	30	/api/v1/admin	User mgmt, config, billing	Admin role
Dashboards	15	/api/v1/dashboards	Widgets, layouts, export	Yes (JWT)
Reports	10	/api/v1/reports	Generate, schedule, download	Yes (JWT)
Webhooks	8	/api/v1/webhooks	Configure, test, logs	Yes (API key)

Module	Endpoint Count	Base Path	Key Operations	Auth Required
TOTAL	160	-	-	-

Table 8.1: API Endpoint Catalog by Module

9. Administrator Guide

The Administrator Guide targets personnel responsible for deploying, configuring, and maintaining CyberSOC Platform installations. Content covers initial installation procedures for various deployment models (cloud-managed, self-hosted, hybrid), configuration parameter reference with security implications noted for sensitive settings, backup and restore procedures ensuring data recoverability, upgrade paths between versions with rollback capabilities, and troubleshooting guidance for common administrative issues. The guide assumes familiarity with Kubernetes concepts, container orchestration fundamentals, and network security principles appropriate for platform administrator roles.

9.1 Administrator Guide Sections

Section	Topics Covered	Target Audience	Pages (est.)
1. Installation	Prerequisites, deployment options, first-run setup	DevOps/SRE	25
2. Configuration	Environment variables, Helm values, feature flags	Platform Admin	40
3. Identity Management	IdP integration, RBAC, SSO configuration	IAM Admin	30
4. Data Management	Database ops, migration, backup/restore	DBA/Platform	35
5. Security Hardening	TLS, network policies, secrets management	Security Eng	45
6. Scaling	HPA tuning, cluster sizing, capacity planning	SRE	25
7. Monitoring Setup	Prometheus, Grafana, alerting configuration	SRE/Ops	30
8. Upgrade Procedures	Version upgrades, migrations, rollback	Release Eng	20
9. Troubleshooting	Common issues, diagnostics, support escalation	Support/Admin	40
10. Maintenance	Scheduled maintenance, patching, health checks	Ops Team	20

Table 9.1: Administrator Guide Structure

10. User Guide

The User Guide serves security analysts, incident responders, and security operations managers who interact with the CyberSOC Platform daily to perform core job functions. Writing style emphasizes task-oriented procedures with clear step-by-step instructions, abundant

screenshots illustrating UI navigation, contextual tips highlighting efficiency shortcuts or common pitfalls, and scenario-based tutorials demonstrating realistic workflows combining multiple features. Accessibility considerations ensure content serves users with diverse abilities including screen reader compatibility for visually impaired analysts and keyboard navigation documentation for users preferring keyboard interaction over mouse-driven interfaces.

10.1 User Guide Learning Paths

Path	Target Role	Modules Covered	Duration	Prerequisites
Quick Start	New analyst	Login, dashboard, basic search	30 min	Account credentials
Threat Hunter	Threat intel analyst	IOC management, feed config, TIP	2 hours	Quick Start
Incident Responder	IR analyst	Case mgmt, evidence, playbook execution	3 hours	Quick Start
SIEM Operator	SIEM analyst	Event search, correlation, alert tuning	2 hours	Quick Start
Administrator	SOC manager	User mgmt, reports, configuration	4 hours	All above
Power User	Advanced analyst	API usage, automation, integration	3 hours	Developer background

Table 10.1: User Guide Learning Paths

11. Knowledge Base Articles

The Knowledge Base (KB) provides searchable, article-based documentation addressing frequently asked questions, known issues with workarounds, how-to guides for specific use cases, and best practice recommendations distilled from field experience. KB articles follow a standardized template including problem statement, affected versions, symptoms, root cause explanation, resolution steps with screenshots, prevention recommendations, and related articles linking to contextually relevant content. Article quality metrics track view counts, helpfulness ratings, and deflection rates (support tickets avoided through self-service) informing content prioritization and improvement efforts.

11.1 Essential KB Categories

Category	Article Count	Top Articles	Update Frequency
Getting Started	25	First login, navigation tour, keyboard shortcuts	Per release
Authentication	18	SSO setup, MFA enrollment, password reset	As needed
Threat Intelligence	32	Feed configuration, IOC formats, TIP usage	Quarterly
Incident Management	28	Case creation, playbook execution, reporting	Monthly
SIEM/Analytics	35	Search syntax, correlation rules, dashboards	Monthly

Category	Article Count	Top Articles	Update Frequency
Integrations	22	SIEM connectors, ticketing, threat feeds	Per integration
Troubleshooting	45	Common errors, performance issues, diagnostics	Weekly
Best Practices	20	Detection tuning, alert triage, reporting	Quarterly
API Usage	15	Authentication, rate limits, examples	Per API change
Security	12	Data privacy, access control, audit logs	As needed

Table 11.1: Knowledge Base Category Structure

12. Release Notes Template

Release notes communicate version changes to customers and internal stakeholders, documenting new features, enhancements, bug fixes, known issues, and migration requirements. Release note quality directly impacts customer satisfaction and support burden; vague entries like "various bug fixes" frustrate customers seeking specific change information while detailed entries enable informed upgrade decisions and efficient troubleshooting following deployment. The template below establishes required sections and formatting standards ensuring consistency across releases while accommodating variation in content volume between major feature releases and minor patch updates.

```
# CyberSOC Platform Release Notes

## Version: [X.Y.Z]
**Release Date:** [YYYY-MM-DD]
**Classification:** [Major / Minor / Patch]
**Upgrade Impact:** [Breaking Changes / Migration Required / Seamless]

---

### 🆕 New Features

| Feature ID | Title | Description | Documentation |
|-----|-----|-----|-----|
| FEAT-001 | [Feature Name] | [Concise description of new capability] | [Link to docs] |

### 🚀 Enhancements

| Enhancement ID | Component | Description |
|-----|-----|-----|
| ENH-001 | [Component] | [Description of improvement] |

### 🐛 Bug Fixes

| Bug ID | Component | Severity | Description |
|-----|-----|-----|-----|
| BUG-001 | [Component] | [Critical/Major/Minor] | [Fix description] |

### 🔒 Security Fixes

| CVE-ID | Component | Severity | Description |
|-----|-----|-----|-----|
```

```

| CVE-XXXX-XXXX | [Component] | [CVSS Score] | [Vulnerability description] |

### 📌 Known Issues

| Issue ID | Description | Workaround | Planned Fix |
|-----|-----|-----|-----|
| KNOWN-001 | [Issue description] | [Workaround steps] | [Version] |

### 📖 Upgrade Instructions

**From version X.Y:** [Migration notes, breaking changes, manual steps required]

**Estimated downtime:** [Minutes/Hours] or **Zero-downtime rollout supported**

### 📄 Related Documentation

- [Installation Guide](link)
- [Migration Guide](link)
- [API Changelog](link)

---

**Need help?** Contact support@cybersoc.io or visit [support portal](link)

```

Listing 12.1: Release Notes Template